Лабораторна робота №1

Управління кодом та організація робочого простору в Git

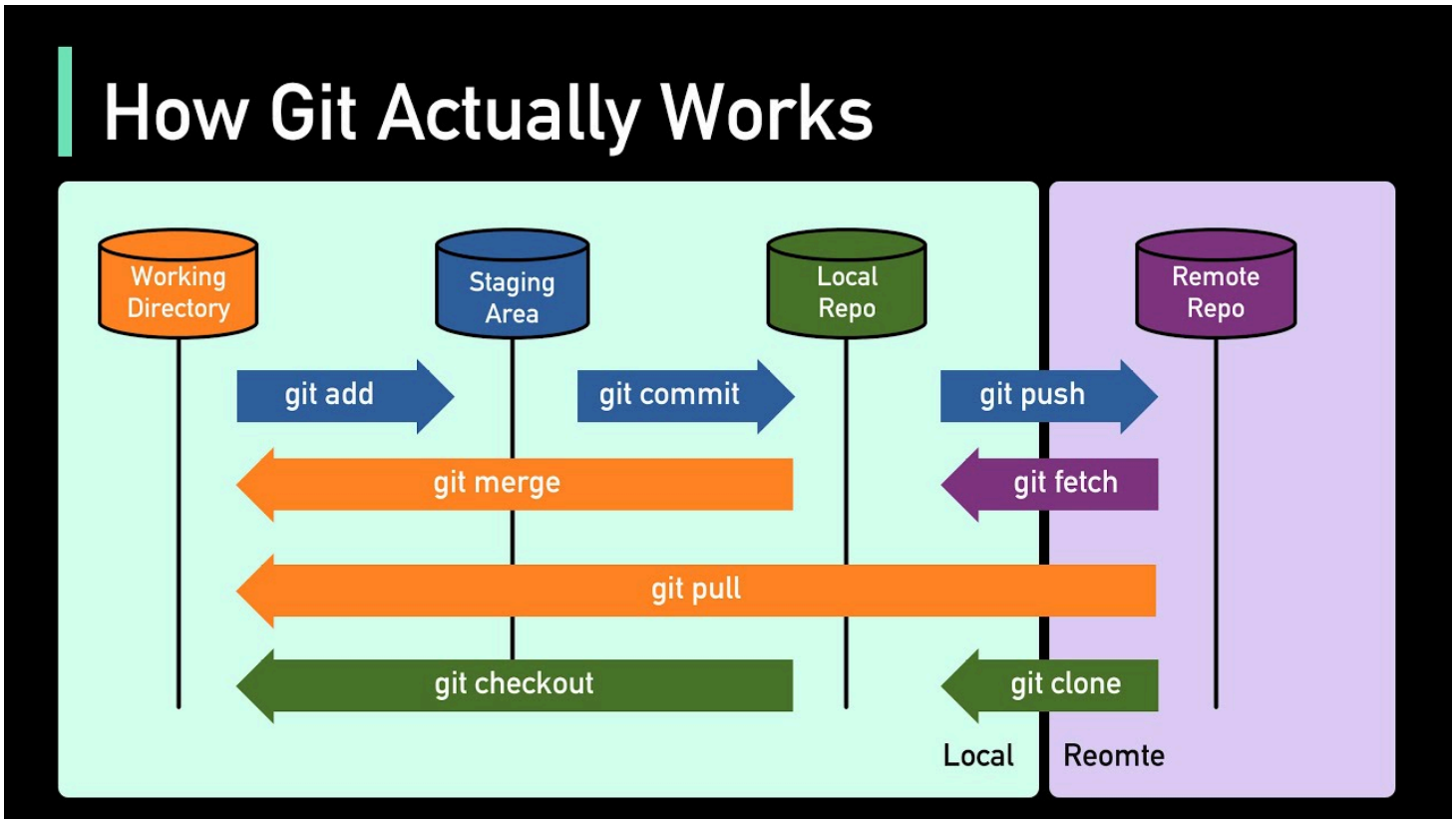
Мета роботи: Опанувати професійні навички роботи з розподіленою системою контролю версій Git. Навчитися організовувати структуру проєкту, застосовувати стратегії розгалуження, працювати з віддаленими репозиторіями (GitHub/GitLab) та впроваджувати процеси спільної розробки через Pull Requests та Code Review.

1. Концептуальна модель Git

Git — це розподілена система контролю версій (DVCS). На відміну від централізованих систем, кожен розробник має повну копію історії проєкту локально. Для ефективного керування змінами Git використовує чотири основні області (стани).

Чотири області (States):

1. **Working Directory (Робоча область):**
   - Це фактична папка на диску вашого комп'ютера, де ви редагуєте код.
   - Файли тут можуть бути Untracked (Git про них ще не знає) або Modified (змінені, але не підготовлені до збереження).
2. **Staging Area / Index (Область підготовки):**
   - Це проміжний шар або "буфер". Сюди файли потрапляють після команди git add.
   - Вона дозволяє групувати зміни: ви можете змінити 10 файлів, але додати в індекс лише 2, які стосуються поточної задачі. Це дозволяє створювати чисті логічні коміти.
3. **Local Repository (Локальний репозиторій):**
   - База даних у прихованій папці .git.
   - Після виконання git commit зміни зберігаються тут як незмінний "знімок" (snapshot) історії з унікальним хешем.
4. **Remote Repository (Віддалений репозиторій):**
   - Версія проєкту, що зберігається на сервері (GitHub, GitLab).
   - Слугує центром синхронізації коду між усіма членами команди.



Основні команди Git

Для повсякденного робочого циклу використовується наступний набір команд:

Ініціалізація та синхронізація

- **git init** — створення нового репозиторію в поточній папці.
- **git clone <url>** — копіювання існуючого репозиторію з сервера на локальну машину.
- **git remote add origin <url>** — прив'язка локального репозиторію до віддаленого сервера.

Робочий цикл (Daily Workflow)

- **git status** — перевірка стану файлів (що змінено, що в індексі).
- **git add <file> або git add .** — переміщення змін до Staging Area.
- **git commit -m "prefix: description"** — фіксація змін у локальній історії. Рекомендовано Conventional Commits (feat:, fix:, docs:).
- **git push origin <branch>** — відправка локальних комітів на віддалений сервер.
- **git pull** — отримання змін із сервера та їх автоматичне злиття з локальною гілкою.

Аналіз та гілки

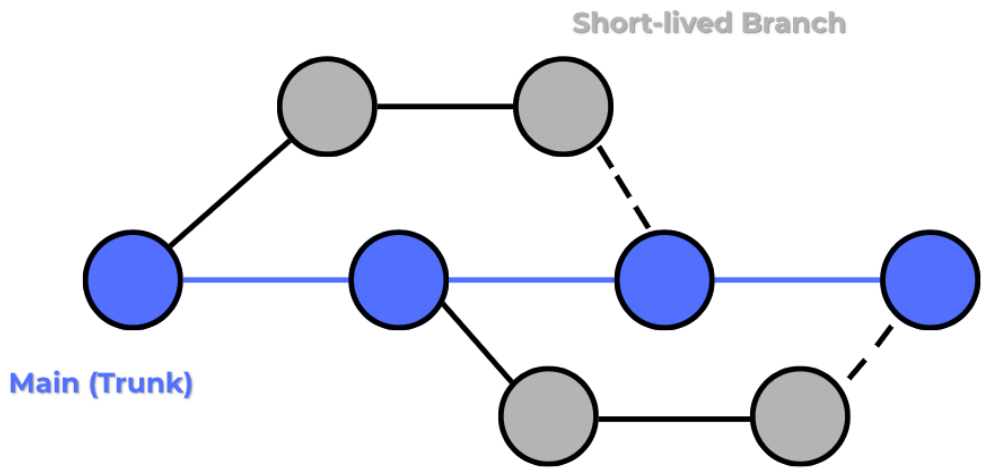
- **git log --oneline --graph** — перегляд історії комітів у вигляді дерева.
- **git branch** — перегляд списку існуючих гілок.

- **git checkout -b <name>** — створення нової гілки та миттєвий перехід на неї.
- **git merge <branch>** — об'єднання вказаної гілки з поточною.

Стратегії розгалуження (Branching Strategies)

Trunk-based development

- **Концепція:** Усі розробники працюють у єдиній головній гілці (main).
- **Особливість:** Коміти маленькі та дуже часті. Фічі приховуються за "Feature Flags".

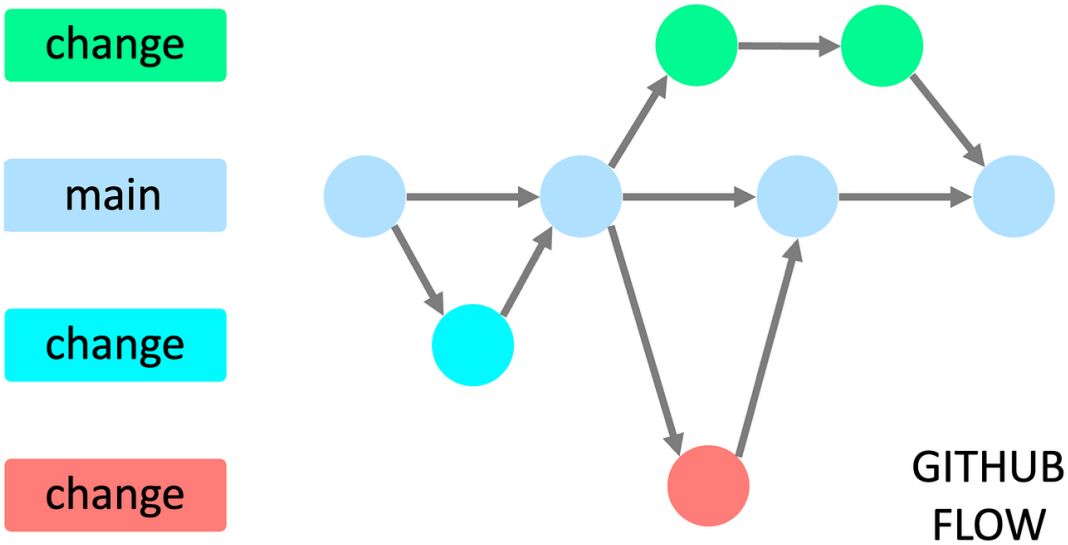


GitFlow (Класичний підхід)

- **main / master** — стабільний код на продакшені.
- **develop** — гілка інтеграції для наступного релізу.
- **feature/*** — тимчасові гілки для окремих завдань.
- **release/*** — стабілізація перед випуском версії.
- **hotfix/*** — термінові виправлення багів на продакшені.

GitHub Flow

- Будь-яка зміна — окрема гілка.
- Створення **Pull Request** для обговорення.
- Після Code Review — злиття в main та негайний деплой.

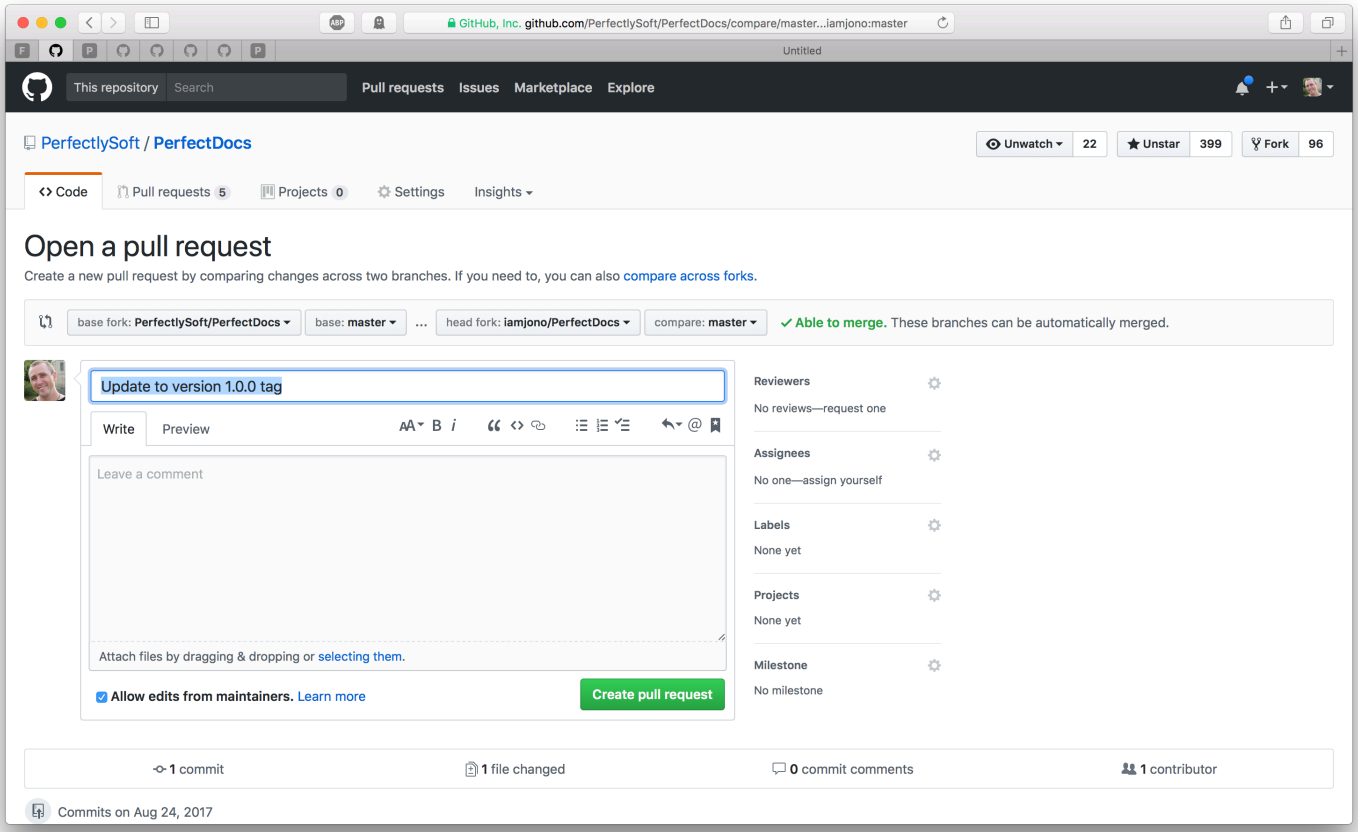


Командна робота з Git

У професійному середовищі розробники ніколи не пушать код напряму в основну гілку. Для цього використовується механізм запитів на злиття.

Створення Pull Requests (PR) / Merge Requests (MR)

- **Концепція:** PR (на GitHub/Bitbucket) або MR (на GitLab) — це пропозиція внести зміни з вашої гілки фічі (напр. feature/task-logic) до базової гілки (напр. develop або main).
- **Процес:**
  i. Розробник завершує роботу в локальній гілці.
  ii. Пушить гілку на сервер.
  iii. Відкриває PR, описуючи, які зміни було внесено та які проблеми вони вирішують.
- **Переваги:** Це створює "ізоляційний шар", де код можна перевірити, протестувати та обговорити перед тим, як він стане частиною продукту.



Основи Code Review

Code Review — це процес перевірки коду одного розробника іншим.

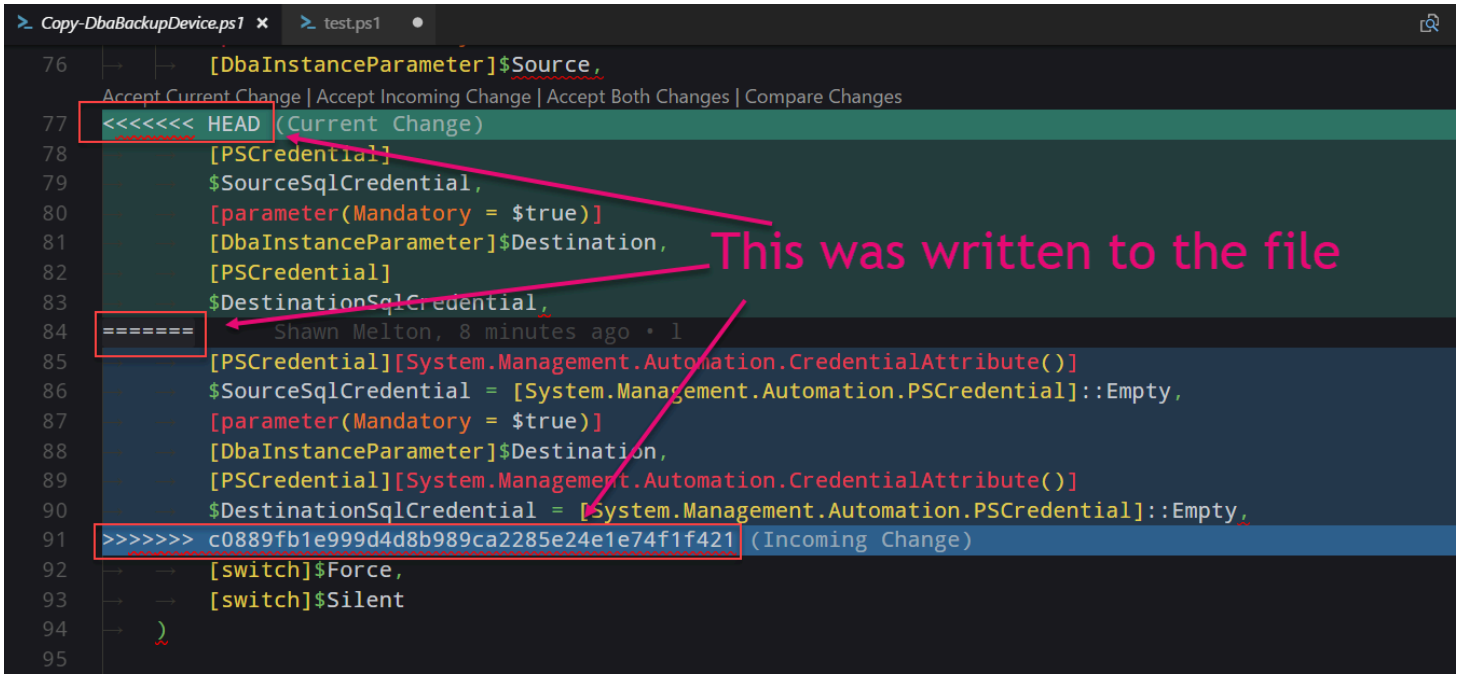
- **Цілі:**

    - **Виявлення багів:** Око колеги може помітити помилку, яку пропустив автор.
    - **Обмін знаннями:** Команда знає, як працюють частини системи, написані іншими.
    - **Дотримання стандартів:** Перевірка відповідності коду архітектурним правилам та стилю.
- **Правило "без его":** Критика має стосуватися коду, а не особистості розробника. Менеджеру важливо, щоб рев'ю було конструктивним та швидким.

Вирішення конфліктів злиття (Merge Conflicts)

Конфлікт виникає, коли Git не може автоматично об'єднати два коміти (наприклад, два розробники змінили один і той самий рядок в одному файлі).

- **Алгоритм вирішення:**
    i. Git зупиняє процес злиття та позначає конфліктні файли.
    ii. Розробник відкриває файл і бачить спеціальні маркери:
    <<<<<<< HEAD (ваш код)
    ======= (розділювач)
    >>>>>>> branch-name (код, що вливається).
    iii. Потрібно вручну обрати правильну версію коду або поєднати обидві.
    iv. Після виправлення — git add та завершення коміту злиття.



Гігієна репозиторію

Чистота репозиторію напряму впливає на безпеку та швидкість роботи команди.

Робота з .gitignore

Файл .gitignore вказує Git, які файли та папки НЕ потрібно відстежувати.

- **Що обов'язково ігноруємо:**
  - **Залежності:** node_modules/ (вони займають гігабайти і легко відновлюються).
  - **Логи та тимчасові файли:** *.log, .DS_Store.
  - **Секрети:** Файли .env (паролі, ключі API). Злиття секретів у публічний репозиторій — критична помилка безпеки.
  - **Артефакти збірки:** Папка dist/ або build/.

.gitignore

```
# IDE's project files
.idea

# windows thumbnail cache
Thumbs.db

# composer vendor dir
/vendor

# Mac DS_Store Files
.DS_Store

# Large output files
build/output/*

# machine-specific configuration files
.env
config/dev-env.conf

```

Написання якісних Commit Messages (Conventional Commits)

Для того, щоб історія проєкту була зрозумілою менеджеру та автоматичним інструментам, використовується стандарт **Conventional Commits**.

- **Структура:** <type>(<scope>): <description>
- **Основні типи (types):**
  - feat: — додавання нової функціональності.
  - fix: — виправлення багу.
  - docs: — зміни в документації.
  - refactor: — зміна коду без зміни функціоналу.
  - test: — додавання або виправлення тестів.
- **Приклад:** feat(ui): add priority tags to task cards

Документування проєкту

Роль README.md як "обличчя" продукту

Файл README.md — це перше, що бачить будь-який розробник або інвестор, відкриваючи репозиторій.

- **Чому це важливо:**
  - **Швидкий старт:** Інструкція з інсталяції економить години часу новим учасникам.
  - **Маркетинг:** Опис проєкту пояснює його цінність (Vision).
  - **Status:** Тут зазвичай розміщують бейджі (Build Status, Test Coverage).
- **Ідеальна структура:**
  i. Назва та короткий опис (One-liner).
  ii. Стек технологій.
  iii. Інструкція з розгортання (npm install, npm run dev).
  iv. Посилання на Live Demo (якщо є).

2. Індивідуальне завдання

Крок 1: Ініціалізація та налаштування

- Створення віддаленого репозиторію.
- Налаштування локального оточення (git config).
- Створення файлу .gitignore, адаптованого під обраний стек (Frontend/Backend).

Крок 2: Первинна структура проєкту (MVP Setup)

- Створення базової структури папок.
- Написання README.md (назва проєкту, опис ідеї MVP, інструкція з запуску).
- Перший коміт у гілку main.

Крок 3: Робота за стратегією Feature Branch

- Створення гілки develop (якщо обрано GitFlow).
- Створення персональної гілки для нової фічі (наприклад, feature/initial-layout).
- Імітація розробки (створення декількох файлів).

Крок 4: Спільна робота та Code Review

- Пуш гілки у віддалений репозиторій.
- Відкриття Pull Request.
- Завдання для пари студентів або студента та викладача: проведення рев'ю, написання коментарів до коду, виправлення зауважень.

Крок 5: Вирішення конфліктів (Обов'язково)

- Спеціальне створення конфлікту в одному файлі через різні гілки.
- Ручне вирішення конфлікту та фінальний мердж.

3. Вимоги до звіту

1. Посилання на публічний репозиторій.
2. Скриншот графіка гілок (Network graph), що демонструє коректне розгалуження.
3. Посилання на закритий Pull Request з коментарями Code Review.
4. Обов'язково присутній файл .gitignore і README

4. Контрольні запитання

1. У чому різниця між git merge та git rebase з точки зору історії проєкту?
2. Чому не можна зберігати файли конфігурації з паролями (.env) у репозиторії?
3. Яку роль відіграє Pull Request у забезпеченні якості продукту (QA)?